

2026-08-10 Update: Three-Signal Ablation, Steering Research

1. Gate Three-Signal Ablation — Completed

The goal is to compare "local signal only" (Δh , gate v2), "global signal only" (h , h-only), and "feature-level fusion of both" (gate v3, the main method) under one controlled protocol. Full data is in [gate_v3_main_method_report.md](#) appendix B.2.

1.1 Gate's Own Classification Ability (offline, dev set)

All three share the same layer choice and pooling protocol ($j=0,1 \rightarrow$ layer 15, $j=2,3 \rightarrow$ layer 23); the only variable is the input feature:

Variant	Feature	Dev AUC	Overall F1	Overall TPR
Local signal only (gate v2)	Δh_j	0.9221	0.7785	0.8456
Global signal only (h-only)	h_j	0.9339	0.7939	0.876
Feature-level fusion (gate v3)	$h_j \oplus \Delta h_j$	0.9357	0.793	0.8838

Trained on h_j alone, classification quality is already very close to the fusion model — overall F1 is even marginally higher than fusion (within noise) — indicating h_j is a stronger standalone signal than Δh_j .

1.2 End-to-End Results (three datasets, unified rawprefix + $\lambda=0.50$)

All four arms (A/B/C/D) share one configuration: [bart_decompose](#), rawprefix (the accumulated prefix fed to the gate for scoring uses raw, un-expanded sub-questions, matching the training-time text distribution), $\lambda=0.50$, BGE's query-side retrieval instruction prefix on the retrieval side, and real recall@1/recall@3 (computed over the full candidate pool before beam-width truncation, not subject to the `--beam-width 1` degeneracy). B, C, and D differ ONLY in which gate signal is used — a clean single-variable comparison.

MuSiQue (2,417 examples)

Arm	recall@1	recall@3	chain@1	EM	F1
A. Cosine similarity only (no gate)	0.6003	0.7611	0.3335	0.3757	0.4652
B. Local signal only (gate v2)	0.6846	0.8435	0.4775	0.4150	0.5105
C. Global signal only (h-only)	0.6977	0.8433	0.5077	0.4245	0.5216
D. Feature-level fusion (gate v3)	0.7023	0.8446	0.5143	0.4307	0.5270

2WikiMultihopQA (12,576 examples)

Arm	recall@1	recall@3	chain@1	EM	F1
A. Cosine similarity only (no gate)	0.8123	0.9021	0.6047	0.4544	0.5230
B. Local signal only (gate v2)	0.9119	0.9813	0.8255	0.4912	0.5661
C. Global signal only (h-only)	0.9244	0.9839	0.8515	0.4967	0.5733
D. Feature-level fusion (gate v3)	0.9289	0.9835	0.8613	0.4990	0.5759

HotpotQA (7,405 examples)

Arm	recall@1	recall@3	chain@1	EM	F1
A. Cosine similarity only (no gate)	0.5746	0.7760	0.3361	0.4918	0.6163
B. Local signal only (gate v2)	0.6557	0.8791	0.5350	0.5175	0.6462
C. Global signal only (h-only)	0.6573	0.8671	0.5449	0.5183	0.6478
D. Feature-level fusion (gate v3)	0.6596	0.8685	0.5568	0.5217	0.6517

Conclusion: Across all three datasets, D (feature-level fusion) leads on the four main metrics — recall@1, chain@1, EM, F1. On recall@3, B or C occasionally edge it out by a negligible margin (≤ 0.002 , within noise). More notably, **C (global signal h alone) beats B (local signal Δh alone) on almost every one of the four main metrics**, with HotpotQA's recall@3 as the one exception — consistent with the offline classification results in 1.1: h_j alone is already a stronger signal than Δh_j alone, and the marginal gain from fusing in Δh_j on top of h-only is smaller than the gain from moving from Δh -only to h-only in the first place.

2. Steering Research — Completed

Full report: [update_doc/steering_research_report.md](#). Key conclusions:

- **No new training needed for the direction vector:** projecting any existing gate's (v2 / h-only / v3) `LogisticRegression.coef_` back into the 4096-dim hidden space directly yields a usable steering direction. The dict keys and projection formula for all three artifact types are given in section 3.1 of the report.
- **The core engineering constraint:** the gate-scoring model runs on HF `transformers` (trivial to hook, but it doesn't generate any text), while the model that actually generates the reasoning-chain text is vLLM (whose v1 engine, with its CUDA-graph-compiled PagedAttention, does not support a standard forward hook). This constraint determines the priority order of viable implementation paths.
- **Recommended first step (hours, no GPU generation, zero engineering):** add the direction vector to hidden states already known to be "should be flagged as anomalous" (from counterfactual/negative trace examples) and check whether the gate's own score systematically shifts toward "normal." If this doesn't hold, the more complex work on the generation side isn't worth pursuing — this should be the first checkpoint before committing to anything further.
- **Follow-up path:** once the direction is validated, the smallest next step is to fall back to HF `transformers.generate()` (with the direction vector injected) only for hops the gate

judges high-risk, leaving everything else on vLLM. Hooking into vLLM internally, or training a dedicated SAE/CAA direction, are both heavier options to be considered only after the direction itself is proven useful.

- The strength schedule borrows from 2025-era "adaptive-strength steering" work: use the gate's own existing continuous probability to set intervention strength, with no need to train a separate adaptive-strength module.